In [2]:
```
# ============================================================================
# UNSUPERVISED LEARNING CLUSTERING PIPELINE
# Algorithms: K-Means (Partitioning) & DBSCAN (Density-Based)
# Dataset: Teen Mental Health Dataset
# ============================================================================

import pandas as pd
import numpy as np
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, calinski_harabasz_score

warnings.filterwarnings("ignore", category=UserWarning, module="sklearn")
sns.set_theme(style="whitegrid")

# ============================================================================
# PHASE 1: LOAD DATASET
# ============================================================================
print("=" * 80)
print("PHASE 1: LOADING DATASET")
print("=" * 80)

df = pd.read_csv("Teen_Mental_Health_Dataset.csv")
print("-> Dataset loaded successfully.")
print(f"-> Original Dataset Size: {df.shape[0]} rows, {df.shape[1]} columns")


# ============================================================================
# REQUIREMENT 1: DATA PREPROCESSING AND SAMPLING PROCEDURES
# ============================================================================
print("\n" + "-" * 50)
print("REQUIREMENT 1: DATA PREPROCESSING & DATA CLEANING")
print("-" * 50)

missing_before = df.isnull().sum().sum()
print(f" -> Missing Value Report (BEFORE CLEANING): Found {missing_before} missing

# 1. Duplicate Verification
duplicates = df.duplicated().sum()
print(f" -> Duplicate check: Found {duplicates} duplicate rows.")
if duplicates > 0:
    df.drop_duplicates(inplace=True)
    print("    Justification: Duplicates removed to prevent artificial density clus

# 2. Text Normalization & Standardisation
df['social_interaction_level'] = df['social_interaction_level'].str.strip().str.low
df['platform_usage'] = df['platform_usage'].str.strip().replace({'Tik Tok': 'TikTok
print(" -> Text standardization completed (white spaces trimmed, case alignment).")

# 3. Categorical Feature Encoding
```

```
df['gender'] = df['gender'].map({'male': 0, 'female': 1})
df['social_interaction_level'] = df['social_interaction_level'].map({'low': 0, 'med
platform_dummies = pd.get_dummies(df['platform_usage'], prefix='platform', dtype=in
df = pd.concat([df, platform_dummies], axis=1)
df.drop(columns=['platform_usage'], inplace=True)
print(" -> Encoding completed (Binary Mapping, Ordinal Scale, One-Hot Multi-class).

# 4. Feature Selection & Extraction
if 'depression_label' in df.columns:
    X = df.drop(columns=['depression_label'])
    print(" -> Feature Selection: Removed 'depression_label' target column.")
else:
    X = df.copy()

# Sampling Procedure Explanation
# Justification: Because our dataset scale (1,200 samples) is fully computationally
# manageable in-memory, we utilize 100% of the observations to ensure the density
# structures required for DBSCAN are completely preserved without downsampling bias
print(" -> Sampling Action: Preserved 100% of dataset records to maintain density s

# 5. Missing Value Handling
X = X.fillna(X.mean(numeric_only=True))
missing_after = X.isnull().sum().sum()
print(f" -> Missing Value Report (AFTER CLEANING): Found {missing_after} missing da

# 6. Feature Scaling & Normalization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print(" -> Feature Scaling: Standardization completed (Mean=0, Variance=1).")

# 7. Dimensionality Reduction (PCA)
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
variance = sum(pca.explained_variance_ratio_)
print(f" -> PCA Dimension Reduction: {X.shape[1]} features reduced to {X_pca.shape[
print(f"    Variance Preserved: {variance*100:.2f}%")


# ============================================================================
# MODEL 1: K-MEANS CLUSTERING
# ============================================================================
print("\n" + "=" * 80)
print("MODEL 1: K-MEANS CLUSTERING")
print("=" * 80)

best_k = 2
best_kmeans_score = -1

for k in range(2, 10):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_pca)
    score = silhouette_score(X_pca, labels)
    print(f"   Tested K = {k} | Silhouette Score = {score:.4f}")

    if score > best_kmeans_score:
        best_kmeans_score = score
```

```
        best_k = k

kmeans_model = KMeans(n_clusters=best_k, random_state=42, n_init=10)
kmeans_labels = kmeans_model.fit_predict(X_pca)

kmeans_inertia = kmeans_model.inertia_
kmeans_centers = kmeans_model.cluster_centers_
kmeans_distribution = pd.Series(kmeans_labels).value_counts().sort_index()

print(f"\n-> Optimal K Value Found: {best_k}")
print(f"-> K-Means Silhouette Score: {best_kmeans_score:.4f}")
print(f"-> K-Means Model Inertia: {kmeans_inertia:.4f}")

print("\n-> Cluster Size Distribution:")
for cluster_id, size in kmeans_distribution.items():
    print(f"   Cluster {cluster_id}: {size} samples ({size/len(X_pca)*100:.1f}%)")

print("\n-> Cluster Centers (PCA Coordinate Space):")
for i, center in enumerate(kmeans_centers):
    print(f"   Cluster {i} Center Point: PC1 = {center[0]:.4f}, PC2 = {center[1]:.4


# ============================================================================
# MODEL 2: DBSCAN CLUSTERING
# ============================================================================
print("\n" + "=" * 80)
print("MODEL 2: DBSCAN CLUSTERING")
print("=" * 80)

best_dbscan_score = -1
best_eps = None
best_min_samples = None
best_dbscan_labels = None

# Automatic parameter optimization search for eps and min_samples
for eps in np.arange(0.1, 1.0, 0.05):
    for min_samples in range(3, 10):
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        labels = dbscan.fit_predict(X_pca)

        mask = labels != -1
        number_of_clusters = len(set(labels[mask]))

        if number_of_clusters > 1:
            score = silhouette_score(X_pca[mask], labels[mask])

            if score > best_dbscan_score:
                best_dbscan_score = score
                best_eps = eps
                best_min_samples = min_samples
                best_dbscan_labels = labels

if best_dbscan_labels is not None:
    dbscan_labels = best_dbscan_labels
    num_dbscan_clusters = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else
    noise_points = np.sum(dbscan_labels == -1)
```

```
    dbscan_distribution = pd.Series(dbscan_labels).value_counts().sort_index()
else:
    dbscan_labels = np.zeros(len(X_pca)) - 1
    num_dbscan_clusters = 0
    noise_points = len(X_pca)
    dbscan_distribution = pd.Series([-1]).value_counts()

print(f"-> Best eps: {best_eps:.2f}")
print(f"-> Best min_samples: {best_min_samples}")
print(f"-> DBSCAN Silhouette Score (excluding noise): {best_dbscan_score:.4f}")
print(f"-> Identified Clusters Count: {num_dbscan_clusters}")
print(f"-> Total Flagged Outliers / Noise Points (-1): {noise_points} ({(noise_poin

print("\n-> DBSCAN Cluster Size Distribution:")
for cluster_id, size in dbscan_distribution.items():
    label_name = "Noise/Outliers" if cluster_id == -1 else f"Cluster {cluster_id}"
    print(f"   {label_name}: {size} samples ({size/len(X_pca)*100:.1f}%)")


# ============================================================================
# MODEL COMPARISON TABLE
# ============================================================================
print("\n" + "=" * 80)
print("MODEL COMPARISON ANALYSIS")
print("=" * 80)

summary_data = {
    "Algorithm Engine": ["K-Means (Partitioning)", "DBSCAN (Density-Based)"],
    "Optimized Parameters": [f"K={best_k}", f"eps={best_eps:.2f}, min_samples={best
    "Silhouette Score": [f"{best_kmeans_score:.4f}", f"{best_dbscan_score:.4f}"],
    "Clusters Discovered": [str(best_k), str(num_dbscan_clusters)],
    "Noise / Outliers Detected": ["0 (Unsupported by design)", str(noise_points)],
    "Rows Processed": [len(X_pca), len(X_pca)]
}

summary_df = pd.DataFrame(summary_data)
print(summary_df.to_string(index=False))


# ============================================================================
# CLUSTER PROFILING (K-Means Mean Analysis)
# ============================================================================
print("\n" + "=" * 80)
print("CLUSTER PROFILING (K-MEANS PORTRAIT)")
print("=" * 80)

df['KMeans_Cluster'] = kmeans_labels
numeric_cols = df.select_dtypes(include=[np.number]).columns
profile_df = df[numeric_cols].groupby('KMeans_Cluster').mean()

print("Mean Feature Values per K-Means Cluster:")
print(profile_df.T)


# ============================================================================
# VISUALIZATIONS GENERATION (FOUR REQUIRED CHARTS)
```

```
# ======================================================================
print("\n" + "=" * 80)
print("GENERATING VISUALIZATIONS")
print("=" * 80)

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# 1. K-Means Scatter Plot (PC1 vs PC2)
axes[0, 0].scatter(X_pca[:, 0], X_pca[:, 1], c=kmeans_labels, cmap='viridis', alpha
axes[0, 0].scatter(kmeans_centers[:, 0], kmeans_centers[:, 1], c='red', s=150, mark
axes[0, 0].set_title(f"1. K-Means Clustering Space (K={best_k})", fontsize=12, font
axes[0, 0].set_xlabel("Principal Component 1")
axes[0, 0].set_ylabel("Principal Component 2")
axes[0, 0].legend()

# 2. DBSCAN Scatter Plot (PC1 vs PC2)
axes[0, 1].scatter(X_pca[:, 0], X_pca[:, 1], c=dbscan_labels, cmap='plasma', alpha=
axes[0, 1].set_title(f"2. DBSCAN Clustering Space (eps={best_eps:.2f}, min_samples=
axes[0, 1].set_xlabel("Principal Component 1")
axes[0, 1].set_ylabel("Principal Component 2")

# 3. Silhouette Score Comparison Bar Chart
models = ['K-Means', 'DBSCAN']
scores = [best_kmeans_score, best_dbscan_score]
axes[1, 0].bar(models, scores, color=['#1e6091', '#7209b7'], width=0.4)
axes[1, 0].set_ylabel('Silhouette Score')
axes[1, 0].set_ylim(0, 1.1)
axes[1, 0].set_title('3. Silhouette Score Model Comparison', fontsize=12, fontweigh
for i, v in enumerate(scores):
    axes[1, 0].text(i, v + 0.02, f"{v:.4f}", ha='center', va='bottom', fontweight='

# 4. Cluster Size Bar Chart (Combined View of Sizes and Noise)
all_labels_km = [f"KM-C{i}" for i in range(best_k)]
all_sizes_km = [kmeans_distribution[i] for i in range(best_k)]

all_labels_db = [f"DB-C{i}" if i != -1 else "DB-Noise" for i in dbscan_distribution
all_sizes_db = list(dbscan_distribution.values)

combined_labels = all_labels_km + all_labels_db
combined_sizes = all_sizes_km + all_sizes_db
colors = ['#1a759f'] * len(all_labels_km) + ['#5c0d99'] * len(all_labels_db)

axes[1, 1].bar(combined_labels, combined_sizes, color=colors, alpha=0.8)
axes[1, 1].set_ylabel('Number of Samples')
axes[1, 1].set_title('4. Cluster Size and Outlier Distributions', fontsize=12, font
plt.xticks(rotation=15)
for i, v in enumerate(combined_sizes):
    axes[1, 1].text(i, v + 10, str(v), ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.savefig("clustering_comparative_report.png", dpi=300)
print("-> Visualizations saved successfully as 'clustering_comparative_report.png'.


# ======================================================================
# SAVE CLEANED DATASET
```

```
# ============================================================================
df.to_csv("Teen_Mental_Health_Clean_Clustering.csv", index=False)
print("\n-> Cleaned clustering dataset successfully saved.")

print("\n" + "=" * 80)
print("PIPELINE COMPLETE - READY FOR SUBMISSION")
print("=" * 80)
```

```
================================================================================
PHASE 1: LOADING DATASET
================================================================================
-> Dataset loaded successfully.
-> Original Dataset Size: 1200 rows, 13 columns

------------------------------------------------
REQUIREMENT 1: DATA PREPROCESSING & DATA CLEANING
------------------------------------------------
 -> Missing Value Report (BEFORE CLEANING): Found 0 missing data cells.
 -> Duplicate check: Found 0 duplicate rows.
 -> Text standardization completed (white spaces trimmed, case alignment).
 -> Encoding completed (Binary Mapping, Ordinal Scale, One-Hot Multi-class).
 -> Feature Selection: Removed 'depression_label' target column.
 -> Sampling Action: Preserved 100% of dataset records to maintain density structure
s.
 -> Missing Value Report (AFTER CLEANING): Found 0 missing data cells.
 -> Feature Scaling: Standardization completed (Mean=0, Variance=1).
 -> PCA Dimension Reduction: 14 features reduced to 2 components.
    Variance Preserved: 21.70%

================================================================================
MODEL 1: K-MEANS CLUSTERING
================================================================================
   Tested K = 2 | Silhouette Score = 0.5885
   Tested K = 3 | Silhouette Score = 0.8795
   Tested K = 4 | Silhouette Score = 0.7182
   Tested K = 5 | Silhouette Score = 0.5424
   Tested K = 6 | Silhouette Score = 0.3762
   Tested K = 7 | Silhouette Score = 0.3780
   Tested K = 8 | Silhouette Score = 0.3714
   Tested K = 9 | Silhouette Score = 0.3559

-> Optimal K Value Found: 3
-> K-Means Silhouette Score: 0.8795
-> K-Means Model Inertia: 91.4008

-> Cluster Size Distribution:
   Cluster 0: 411 samples (34.2%)
   Cluster 1: 391 samples (32.6%)
   Cluster 2: 398 samples (33.2%)

-> Cluster Centers (PCA Coordinate Space):
   Cluster 0 Center Point: PC1 = 1.6842, PC2 = -0.1378
   Cluster 1 Center Point: PC1 = -0.7509, PC2 = 1.5770
   Cluster 2 Center Point: PC1 = -1.0016, PC2 = -1.4070

================================================================================
MODEL 2: DBSCAN CLUSTERING
================================================================================
-> Best eps: 0.10
-> Best min_samples: 9
-> DBSCAN Silhouette Score (excluding noise): 0.8851
-> Identified Clusters Count: 3
-> Total Flagged Outliers / Noise Points (-1): 45 (3.8%)
```

```
-> DBSCAN Cluster Size Distribution:
   Noise/Outliers: 45 samples (3.8%)
   Cluster 0: 401 samples (33.4%)
   Cluster 1: 384 samples (32.0%)
   Cluster 2: 370 samples (30.8%)

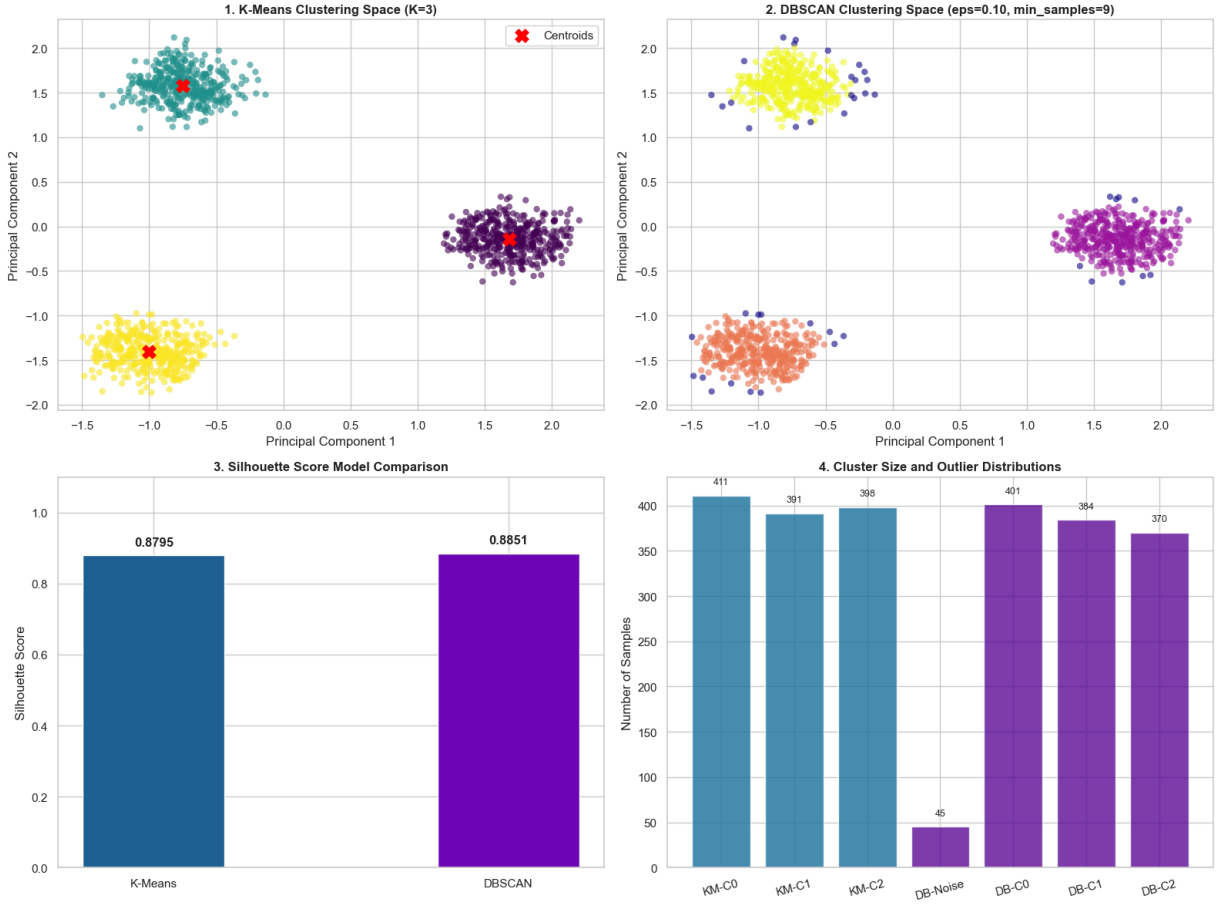
================================================================================
MODEL COMPARISON ANALYSIS
================================================================================
      Algorithm Engine    Optimized Parameters Silhouette Score Clusters Discovered
Noise / Outliers Detected  Rows Processed
K-Means (Partitioning)                     K=3           0.8795                   3
0 (Unsupported by design)            1200
DBSCAN (Density-Based) eps=0.10, min_samples=9           0.8851                   3
45            1200

================================================================================
CLUSTER PROFILING (K-MEANS PORTRAIT)
================================================================================
Mean Feature Values per K-Means Cluster:
KMeans_Cluster                     0          1          2
age                        15.866180  15.956522  15.964824
gender                      0.493917   0.483376   0.484925
daily_social_media_hours    4.559367   4.518414   4.531156
sleep_hours                 6.439659   6.462916   6.446231
screen_time_before_sleep    1.712409   1.745269   1.764322
academic_performance        2.995499   2.975192   3.000025
physical_activity           1.040389   1.020460   0.981910
social_interaction_level    1.038929   0.895141   0.947236
stress_level                5.498783   5.549872   5.288945
anxiety_level               5.669100   5.488491   5.748744
addiction_level             5.581509   5.496164   5.615578
depression_label            0.024331   0.023018   0.030151
platform_Both               0.000000   1.000000   0.000000
platform_Instagram          1.000000   0.000000   0.000000
platform_TikTok             0.000000   0.000000   1.000000

================================================================================
GENERATING VISUALIZATIONS
================================================================================
-> Visualizations saved successfully as 'clustering_comparative_report.png'.

-> Cleaned clustering dataset successfully saved.

================================================================================
PIPELINE COMPLETE - READY FOR SUBMISSION
================================================================================
```


1. K-Means Clustering Space (K=3)
Centroids
Principal Component 1
Principal Component 2
2. DBSCAN Clustering Space (eps=0.10, min_samples=9)
Principal Component 1
Principal Component 2
3. Silhouette Score Model Comparison
0.8795
0.8851
Silhouette Score
K-Means
DBSCAN
4. Cluster Size and Outlier Distributions
411
391
398
45
401
384
370
Number of Samples
KM-C0
KM-C1
KM-C2
DB-Noise
DB-C0
DB-C1
DB-C2

In [ ]: